SECURITY DOCUMENTATION

Security Architecture & Assurance

How MIRQAB is secured, what was tested, and exactly where the evidence ends — every control labelled VERIFIED, LIMITED, BLOCKED or NOT DEPLOYED.

TEAM

Bdoor Fahed Alosaimi
Nouf Saleh Altalhi
Alaa Rafa Al Saad

PROJECT

MIRQAB — AI-Powered Security Operations Platform for SMBs

27 September 2026

Scope and evidence

This document describes the security design of MIRQAB as it exists in the repository on **27 September 2026**. Evidence comes from three sources:

1. **The team's 18-phase security program** (6–8 Sep 2026). Each phase ended with a written report, and several included live tests against the running Docker stack.
2. **The automated test suite**, re-run on 27 Sep 2026: 38 backend suites / 421 tests passing, with clean TypeScript checks for the backend, SOC Dashboard and Employee Desktop.
3. **Direct inspection** of the current code and configuration for this package, including a fresh `npm audit` run on 27 Sep 2026.

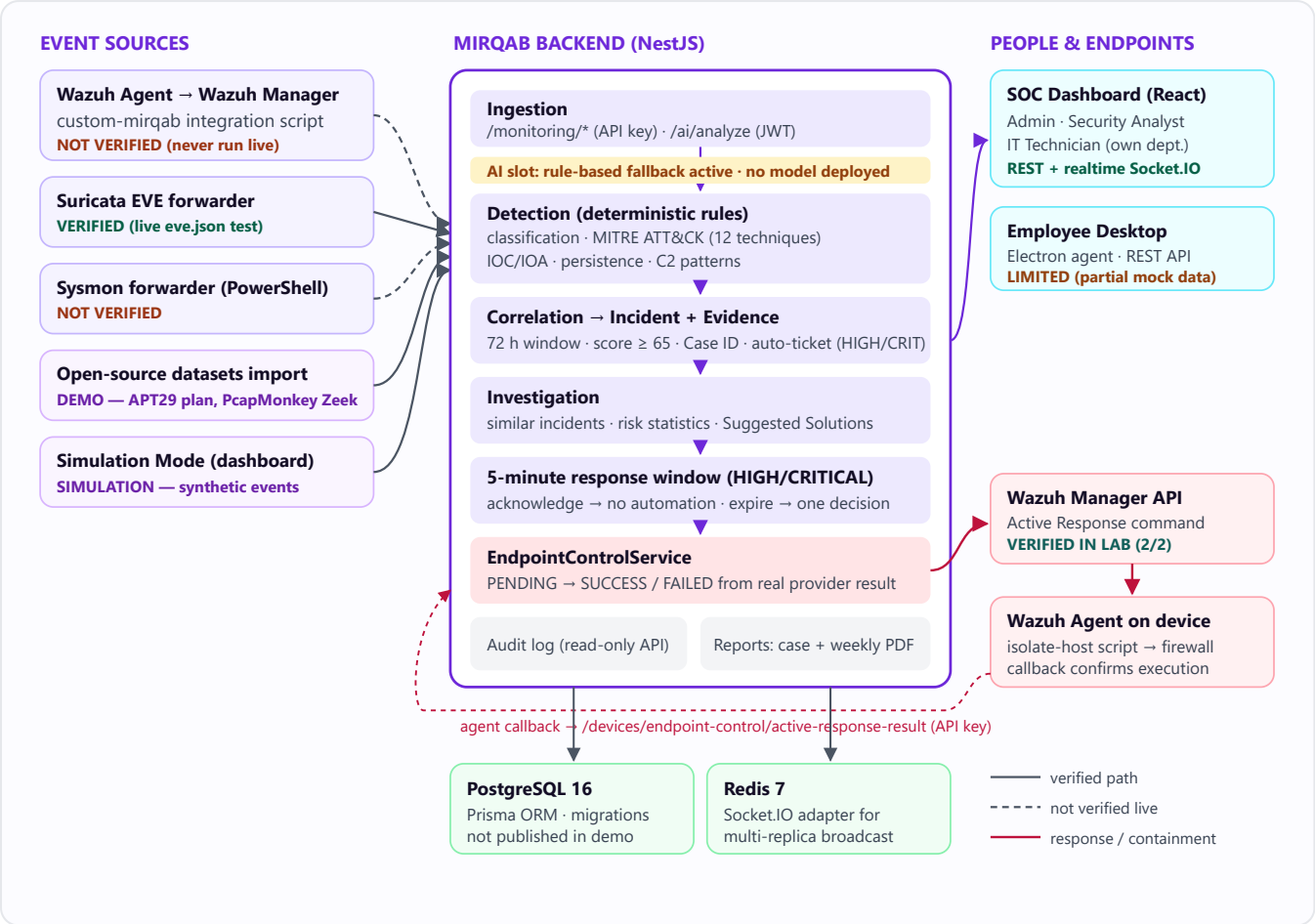
Important

MIRQAB is NOT DEPLOYED to production. No external penetration test was performed. Features added after 8 Sep 2026 (the reworked 5-minute response window, Suggested Solutions, the evaluator demo stack) are covered by unit tests but have not had a dedicated security review.

How to read the status labels

VERIFIED	Tested and confirmed: by the automated test suite (38 suites, 421 tests, all passing on 27 Sep 2026) and/or by a live test recorded in the team's engineering reports.
VERIFIED IN LAB	Confirmed end-to-end in a controlled, containerized lab. Not tested on physical employee devices.
LIMITED	Implemented and working, with a clearly stated limitation.
NOT VERIFIED	Implemented in code but not tested live.
NOT IMPLEMENTED	Not available in the application.
NOT DEPLOYED	No production deployment exists.
BLOCKED	Cannot be completed because of an external dependency.
PLANNED	A future roadmap item. Not built.
DEMO / SIMULATION	Uses synthetic or public demo data. Not a real environment.

1. Security architecture



Trust boundary	Control	Status
Browser → nginx	TLS termination with HTTP → HTTPS redirect (self-signed development certificate), security headers.	VERIFIED LIMITED no CA certificate
nginx → backend	Reverse proxy for `/api/v1` and `/socket.io`. Forwarded-IP handling was reviewed in Phases 12 and 18.	VERIFIED
Dashboard user → API	JWT authentication, role and department/organization authorization, input validation, rate limits.	VERIFIED
Forwarders → API	Shared `X-Api-Key` on `/monitoring/*`, a separate rate limit (1000/min), vendor payload validation.	VERIFIED
Backend → Wazuh Manager	Credentials from server config only. Manager certificate trusted through an explicit CA file.	VERIFIED IN LAB
Agent → backend callback	API key. The reported agent name must match the device awaiting isolation. No network-supplied value reaches a shell command.	VERIFIED IN LAB
Backend → AI provider	Single provider slot, server-configured URL only, fail-closed default, output validation. Currently none active.	VERIFIED

2. Authentication

- Access tokens: JWT, 15-minute lifetime. Verification is **pinned to HS256** at all 4 verification sites (3 REST, 1 WebSocket), which rules out algorithm-confusion attacks.
- Refresh tokens: 7-day lifetime and **rotated on every use**. Reuse of an old refresh token revokes the entire token family.
- Passwords: bcrypt with 12 rounds. The password hash is structurally excluded from every API response.
- Login failures return one generic message ("Invalid email or password"), so accounts cannot be enumerated.
- Password-reset tokens are never logged in production. A reset-token leak and a privileged self-signup path were found and fixed (commit "Fix two auth vulnerabilities").

VERIFIED — security Phase 11 (live against a rebuilt backend) and unit tests (`auth.service.spec.ts`).

3. Authorization and RBAC

Role	Access
`ADMIN`	Organization-wide, including user management, policies and settings
`SECURITY_ANALYST`	Organization-wide investigation and response
`IT_TECHNICIAN`	Own department only (incidents, devices, tickets, realtime rooms)
`EMPLOYEE`	Employee Desktop only. Rejected by the realtime gateway and dashboard routes.

Department scoping lives in one shared helper, used both by the object-level guards (`IncidentAccessGuard` , `TicketAccessGuard` , `DeviceAccessGuard`) and by every collection query. VERIFIED — Phase 1: 18/18 live tests, with no cross-department leak over REST or WebSocket. Phase 16 found and fixed two more department-scope gaps (similar-incident candidates and the case report), which are now covered by dedicated tests.

4. Organization / tenant isolation

Tenant-owned records carry `organizationId` . Every query applies the user's organization, and a user with no organization receives a guaranteed-empty result (fail-closed, never "all organizations"). Guards compare the resource's organization with the user's before the department check. VERIFIED by unit tests (organization scoping appears in 15 test files) · LIMITED : no live test with several organizations is recorded, and this layer (8–9 Sep 2026) came after the Phase 17/18 regression passes.

5. IDOR / BOLA protections

- `ParseUUIDPipe` on every `:id` route. Malformed IDs are rejected before any database access.
- Object-level guards on incident, ticket and device routes, applying department and organization checks.
- The isolation target device is resolved **on the server** from the incident relation, never from the request body. `CreateIncidentActionDto` has no `deviceId` , `hostname` or `performedBy` field.
- For isolation, the request's claimed outcome is always overwritten by the real provider result.

VERIFIED — Phases 1, 5 and 16.

6. Input validation

A global `ValidationPipe` uses `whitelist`, `forbidNonWhitelisted` and `transform`, so unknown fields are rejected. A global pipe rejects null bytes (a live test in Phase 5 had found they caused a 500 error). DTOs carry `@MaxLength` bounds, for example 10–50,000 characters for analysed log excerpts. VERIFIED — Phase 5.

7. API security

- Helmet security headers on the backend. nginx adds `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy` and `Strict-Transport-Security`.
- Swagger/OpenAPI documentation is **disabled when** `NODE_ENV=production` (OWASP API #9, inventory exposure). The Docker stacks run in production mode.
- A global exception filter returns uniform error bodies without stack traces.

VERIFIED — Phase 5 (OWASP API Top 10 review).

8. Rate limiting

Endpoint(s)	Limit
Global default	100 requests / minute
`POST /auth/login`, `/auth/signup`, `/auth/forgot-password`, `/auth/reset-password`	5 / minute each
`POST /ai/analyze`, `POST /ai/incidents/:id/analyst-report`	20 / minute
Endpoint-control routes	100 / minute
`/monitoring/*` (forwarders)	1000 / minute
Socket.IO connections	Per-IP connection rate limiter with a memory bound

VERIFIED · LIMITED: `POST /reports/weekly/generate` has no dedicated throttle beyond the global limit (deferred in Phase 18).

9. Session security

Short-lived access tokens, rotating refresh tokens with replay detection, and a fix for the 401-interceptor deadlock ("checking session" hang). Authentication endpoints are excluded from the retry interceptor in both apps. The Employee Desktop stores its refresh token encrypted through the operating system's secure storage. The realtime connection is forcibly disconnected when its access token expires. VERIFIED — Phases 11, 12 and 13.

10. Socket.IO / realtime security

- The handshake is authenticated with the same HS256-pinned JWT. Only dashboard roles are accepted, and inactive users are rejected.
- There are **no client-to-server message handlers** at all: the channel is server-to-client only.

- Events are routed to department- and organization-scoped rooms, so a technician never receives another department's events.
- Per-IP connection rate limiting, with trusted-proxy handling, and a timed disconnect at token expiry.
- The Redis adapter is a pure cross-instance relay for multi-replica deployments. It has no influence on room membership.

VERIFIED — Phase 12 (30 gateway tests at Phase 18).

11. Audit logging

Security-relevant actions (logins, failed logins, denied access, incident status changes, response actions and their outcomes, policy changes, acknowledgements, response-window decisions) are written with a **server-derived** actor and target. Isolation audit entries record the real outcome (SUCCESS/FAILED). The audit API exposes only `GET`, with no create, update or delete routes. Note that for a failed login the recorded email is the *attempted* identity, not a verified one. VERIFIED — Phase 6.

12. Incident integrity

Correlation merges duplicates into one case instead of multiplying incidents. Case IDs come from an atomic per-day counter. Evidence records its source (*Live Agent*, *Open-Source Dataset*, *Simulation*, *Manual / Test*), so synthetic data can never pass as real telemetry. Response actions carry a server-derived actor, and their outcomes come only from the real provider. VERIFIED — Phase 7.

13. Device isolation security

- Only authorized roles, on incidents within their scope, can request isolation. A confirmation dialog states that only this endpoint is affected.
- The device comes from the incident relation on the server. The action is saved as PENDING first, and the outcome is set only by the provider result.
- `Device.isolated` is computed at read time, only from a genuine SUCCESS action.
- **AI can never isolate:** no AI or agent code path is wired to the endpoint-control service. A test confirms that an AI reply containing `{"action":"ISOLATE_DEVICE"}` creates zero actions.
- High-impact recommended actions require explicit analyst approval before execution.

VERIFIED IN LAB — Phases 2/2A and 14. LIMITED: manual isolation is not blocked for incidents labelled as simulated. It remains an explicit human decision behind the confirmation dialog.

14. Wazuh Active Response

MIRQAB never trusts the Manager's "dispatched" reply on its own. Success requires the agent's callback confirming that the script executed. Phase 2A found and fixed two real script bugs: a standard-input read that could hang `wazuh-execd` (now bounded by a timeout), and a hostname mismatch that made the backend silently drop callbacks. The scripts take no network-supplied value into a shell command (Phase 14).

Item	Status
Linux `isolate-host.sh`, end-to-end through MIRQAB	VERIFIED IN LAB 2/2, 11–15 s
Windows `isolate-host.ps1`	NOT VERIFIED
`unisolate-host.sh` / release action in MIRQAB	NOT IMPLEMENTED in the application; script untested
Wazuh 4.14.7 GA image	BLOCKED — upstream tag served release-candidate builds when checked

15. Secrets management

- Real secrets live only in untracked `.env` files. The repository ships `.env.example` files with placeholders. `*.pem` keys are git-ignored.
- Phase 3 rotated every real secret through its proper mechanism and verified that old values were rejected. A full history search found **no secret committed** to the MIRQAB repository. A nested vendor checkout (`wazuh-docker`) with certificate files was identified, and it is excluded from the submitted source package.
- The evaluator demo refuses to start unless the evaluator supplies their own admin password and random secrets (`:?` checks in Compose). It never uses built-in credentials.
- LIMITED : `backend/.env.example` , `infra/docker/.env.example` , the seed script and two test scripts fall back to the public placeholder password `ChangeMe123!` when none is supplied. It must always be replaced.
- No external secret manager (such as Vault) is used. PLANNED for production.

16. TLS

nginx terminates TLS and redirects HTTP to HTTPS. LIMITED : the certificate is **self-signed** (`generate-self-signed-cert.sh`). A CA-issued certificate needs a public domain, which does not exist (NOT DEPLOYED). The evaluator demo is plain HTTP on `127.0.0.1:8080` only.

17. Container security

- The backend image is multi-stage: the runtime contains only production dependencies, and the container runs as the **non-root** `node` user (Phase 10).
- PostgreSQL and Redis have health checks, and the backend starts only after both are healthy.
- **Evaluator demo:** PostgreSQL and Redis are **not published** to the host, and the gateway binds to `127.0.0.1` only.
- LIMITED — **development stack** (`infra/docker`): PostgreSQL is published on **all host interfaces** (`0.0.0.0:5432` , for a natively-run backend), and the backend container port is published on a random host port on all interfaces. On a shared or untrusted network this exposes the database login prompt. Bind these to `127.0.0.1` or remove them for anything beyond local development. *(Observed on 27 Sep 2026 while preparing this package.)*
- No container image scanner (Trivy, Grype or Docker Scout) and no SBOM tool has been run. NOT VERIFIED

18. Dependency and supply-chain hardening

Phase 10 applied every safe (non-major) `npm audit` fix and verified the builds. The remaining findings need major-version upgrades and were deliberately deferred. A fresh `npm audit` on **27 Sep 2026** reports:

Application	Critical	High	Moderate	Low	Notes (from Phase 10 analysis)
Backend	0	9	15	5	Mostly transitive through NestJS 10 (for example `multer` , which is never wired to a route, and `lodash` , whose vulnerable function is never called). Fix requires a NestJS major upgrade.
SOC Dashboard	0	1	3	0	Build tooling (`vite` / `esbuild`). Fix requires a Vite major upgrade.

Application	Critical	High	Moderate	Low	Notes (from Phase 10 analysis)
Employee Desktop	1	10	3	0	Electron 29 and electron-builder 24 chains. Fix requires major upgrades (Electron 44, electron-builder 26).

LIMITED — known, analysed and documented, but **not remediated**. The Employee Desktop Electron upgrade is the most important pending item.

19. Frontend security

- Zero `dangerouslySetInnerHTML`, `innerHTML`, `eval` or `new Function` in either application. React escaping applies everywhere.
- Zero `console.*` calls in application source. No hard-coded secrets. The dashboard renders no clickable external links.
- Electron: `contextIsolation` on, `nodeIntegration` off, sandbox on, a `will-navigate` guard, and an allow-list of URL schemes for external links.
- When the backend is unreachable, a "stale data" banner stops wallboard views from silently showing outdated information.

VERIFIED — Phase 13.

20. Realtime security

See section 10. The live network map and Citadel views are fed only by department- and organization-scoped events and REST data. Phase 14 re-confirmed every earlier fix in this area. VERIFIED

21. Failure handling

Failure	Behaviour	Status
AI provider unavailable or slow	Timeouts, a circuit breaker and a concurrency cap. Falls back to rules. No silent switch to an external provider.	VERIFIED (Phase 9)
Malformed AI output	Rejected by strict schema and severity validation.	VERIFIED
Wazuh / endpoint provider unavailable	Isolation recorded as FAILED or "unavailable". The device is never marked isolated.	VERIFIED
Notification write failure	Isolated so it cannot break incident creation (fixed in Phase 15).	VERIFIED
Backend unreachable (UI)	Stale-data banner and polling safety net.	VERIFIED
PostgreSQL or Redis unavailable	Analysed statically. Redis is optional for single-instance mode.	LIMITED static only

22. Idempotency

Acknowledgement is idempotent. The containment decision is taken once per incident under a PostgreSQL advisory-lock transaction with a once-only "window expired" marker, which is safe across scheduler ticks and replicas. Case IDs come from an atomic counter. Seed and demo import create only missing items. VERIFIED by unit tests.

23. Production failure modes

Phase 15 reviewed 10 failure scenarios: PostgreSQL, Redis, Wazuh and AI unavailable, Active Response timeout, WebSocket failure, and others. It fixed two findings and documented one theoretical structural risk. Its verdict was

VERIFIED WITH DOCUMENTED LIMITATIONS, because the PostgreSQL, Redis and Wazuh conclusions are static, not live-tested.

> **Observed on 27 Sep 2026 — schema drift on the development database**
>
> While preparing this package, the team found that the development database had older table shapes for `persistence_findings` and `weekly_reports`. Three renamed migrations had been marked "applied" without executing, which made the Incident Investigation panel fail with HTTP errors. It was repaired with an additive, transactional migration (`20260927000000_reconcile_legacy_persistence_findings`) that was tested on throwaway copies (legacy database, current database and a fresh install) before being applied. A database backup was taken first. Fresh installs are unaffected: on them this migration does nothing.

24. Security regression testing

- **Automated:** 38 Jest suites / 421 tests (27 Sep 2026), covering RBAC and department scoping, organization scoping, authentication, JWT pinning, WebSocket security, AI provider behaviour, device isolation, audit logging, input validation, the response window, and every fix from Phases 1–16.
- **Phase 17** (8 Sep): full regression — **no regression found** (293 tests at that time).
- **Security Testing Mode** (in the app): on-demand, **non-destructive** self-checks of the deployed instance (detection, prompt-injection delimiting, output sanitization, object-level authorization, approval gating, Wazuh payload validation). It uses synthetic data and deletes its single transient test incident afterwards.
- **UI QA:** 27 Playwright scripts (`apps/soc-dashboard/e2e`) run manually during development. There is no CI pipeline.

25. Final security acceptance

Phase 18 (8 Sep 2026) concluded: VERIFIED WITH DOCUMENTED LIMITATIONS. No blocking security regression, and the system is acceptable for deployment *in its current fully-deterministic AI mode*, with two carried-forward limitations: Wazuh GA version BLOCKED and self-hosted AI NOT VERIFIED.

Additional caveats added in this package (27 Sep 2026): development-stack port exposure (section 17), unremediated dependency advisories (section 18), no dedicated security review of post-8-September features, no external penetration test, and NOT DEPLOYED.

26. Summary of status by area

Area	Status
Authentication, sessions, RBAC, IDOR/BOLA, input validation, API security, rate limiting	VERIFIED
Organization isolation	VERIFIED (unit) LIMITED (no live multi-tenant test)
Realtime / Socket.IO, audit logging, incident integrity, frontend	VERIFIED
Device isolation / Wazuh Active Response	VERIFIED IN LAB
Wazuh GA version	BLOCKED

Area	Status
Self-hosted AI	NOT VERIFIED NOT DEPLOYED
TLS (CA certificate), production deployment	NOT DEPLOYED
Dependencies, container scanning, dev-stack port exposure	LIMITED
Failure modes, performance	LIMITED (static analysis / no load test)